Parallel Text Search

Zwick, Patrick
patrick.zwick@htwg-konstanz.de

Döbele, Max
max.doebele@htwg-konstanz.de

06 February 2026

Abstract

This paper presents and evaluates several text search algorithms for processing a single large text with multiple query patterns, focusing on their parallelization across different hardware architectures. We demonstrate CPU-based parallelization using OpenMP, achieving a median execution time of 204 milliseconds and a speed-up of 1020% for a text of 82 million characters with the 100 most common English words as queries.

We further explore GPU-based parallelization using OpenCL, which reduces median execution time to 519 milliseconds, corresponding to a speed-up of 717%. By applying heuristics to estimate the total number of matches, we further reduce median runtime to around 319 milliseconds, yielding a speed-up of 1228% compared to the sequential baseline.

In addition, we show how one algorithm can be parallelized across multiple processes using MPI to be distributed over different hosts, achieving a median execution time of 422 milliseconds and a speed-up of 1050%. We also analyze the performance of all approaches when using 100 longer and less common query words.

The source code is available at `https://github.com/KN-PACO/text-search/` in the `src/` directory. Build and usage instructions can be found in `README.md`.

1 Introduction

The goal of this work is to search a single large text for multiple query strings and to return all positions at which each query occurs. This results in the following function signature for a single application of a text search implementation:

```
std::vector<std::vector<size_t>>
find(const std::string &text, const std::vector<std::string> &queries)
```

The function returns a vector of the same size as the input vector of queries. For each query at position n in the input vector, the result vector contains a corresponding vector at position n with all starting indices where that query occurs in the text.

Every implementation presented in this paper is completely self-contained and does not depend on any other implementation. This allows us to provide a command-line tool where the user can select the desired implementation—such as sequential, OpenMP, OpenCL, or MPI – depending on the wanted hardware target.

2 Benchmarking

For testing, we use a collection of up to 98 books from Project Gutenberg[1] [1], which we concatenate into a single text containing over 82 million characters. We perform queries using, at most, the 100 most common English words [2]. Since most of the 100 most common English words are very short, we additionally use a second query set consisting of randomly selected longer English words with lengths between 7 and 10 characters[2].

[1] See `https://github.com/KN-PACO/text-search/tree/main/data` for all used books.
[2] See `https://github.com/KN-PACO/text-search/blob/main/long-words.txt` for the list of long words.

```
std::vector<std::vector<size_t>>
find_std(const std::string &text, const std::vector<std::string> &queries) {
    std::vector<std::vector<size_t>> indices(queries.size());
    for (size_t i = 0; i < queries.size(); ++i) {
        const std::string &query = queries[i];
        auto pos = text.find(query);
        while (pos != std::string::npos) {
            indices[i].push_back(pos);
            pos = text.find(query, pos + 1);
        }
    }
    return indices;
}
```

Listing 1: `std` reference implementation (`std_text_search.cpp`).

Both the number of books and the number of queries can be varied to evaluate performance under different conditions.

Performance benchmarks were conducted on a system equipped with an Intel Core i5-14600K CPU (6 P-cores at 3.5 GHz and 8 E-cores at 2.6 GHz) and an AMD Radeon RX 7800 XT GPU, using GCC 15.2.0 for compilation. For each experiment, we report the median of 20 measurements.

3 Sequential Implementations

We first implemented several purely sequential versions of the algorithm. The objective was not to achieve peak performance through highly optimized or specialized techniques from computer graphics or numerical computing, but rather to identify opportunities for exploiting data-level and task-level parallelism across different hardware targets. These sequential implementations serve as a baseline for analyzing the structure of the computation and for identifying points where the problem can later be decomposed into independent tasks that are well suited to heterogeneous hardware architectures.

3.1 Reference Implementation

To verify correctness, we use a simple baseline implementation, denoted `std`, which relies on `std::string::find()` for substring matching (see Listing 1).

3.2 Candidate Selection and Validation

The first implementation is divided into two distinct phases:

1. Identification of candidate positions for potential occurrences based on a simple filtering criterion. A position is considered a candidate if three characters match: the first character of the query, the middle character, and the last character.

2. Verification of each candidate position to determine whether it constitutes a valid occurrence.

The initial approach, denoted `candidate_v1`, performs candidate identification followed by validation separately for each query (see Listing 2). The vector is cleared after each query and reused for the next, avoiding unnecessary allocations.

3.2.1 Array Pre-Allocation Instead of Dynamic Vector Growth

Since a `std::vector` may need to repeatedly grow its underlying storage and copy existing elements when its capacity is exceeded if no default capacity is specified, this can introduce a significant performance bottleneck, at least for the first query. To avoid this overhead, the `candidate_v2` implementation instead pre-allocates

```cpp
void find_candidates(const size_t text_length, std::vector<size_t> &results,
                     const std::string &text, const std::string &query) {
    const auto query_length = query.length();
    const auto mid = query_length >> 1;
    const auto end = query_length - 1;
    for (size_t i = 0; i < text_length - query_length; ++i) {
        if (text[i] == query[0] && text[i + mid] == query[mid] &&
            text[i + end] == query[end]) {
            results.push_back(i);
        }
    }
}

bool test_candidate(const size_t index, const std::string &text,
                    const std::string &query) {
    return std::memcmp(text.data() + index, query.data(), query.size()) == 0;
}

std::vector<std::vector<size_t>>
find_candidate_v1(const std::string &text,
                  const std::vector<std::string> &queries) {
    std::vector<std::vector<size_t>> indices(queries.size());
    const auto text_length = text.length();
    std::vector<size_t> results;
    for (size_t i = 0; i < queries.size(); ++i) {
        const std::string &query = queries[i];
        find_candidates(text_length, results, text, query);
        for (const auto &result : results) {
            if (test_candidate(result, text, query)) {
                indices[i].push_back(result);
            }
        }
        std::memset(results.data(), 0, results.size() * sizeof(size_t));
    }

    return indices;
}
```

Listing 2: `candidate_v1` implementation (`candidate_v1_text_search.cpp`).

an array with a size equal to the length of the text. In the worst case, every position in the text could be a valid candidate, so this allocation is sufficient to store all candidates.

3.2.2 Using a Bitmask

To improve CPU cache utilization and reduce the memory footprint, the `candidate_v3` implementation represents the candidate buffer as a *bitmask*, where each bit set to 1 denotes a valid candidate position. The bitmask is stored as an array of unsigned 64-bit integers, allowing 64 candidates to be packed into a single word.

Let m denote the length of the text. To represent candidate positions for a query of any length, we require

$$h = \left\lceil \frac{m + 63}{64} \right\rceil$$

64-bit words in the bitmask.

For a candidate at position k, the word index w of the corresponding 64-bit word B_w in the bitmask is obtained by dividing k by 64 which can be efficiently computed by a right shift of 6 bits:

$$w = \frac{k}{64} = k \gg 6.$$

The bit position b within the word B_w is given by taking k modulo 64, which can be efficiently computed by a bitwise AND with 63:

$$b = k \bmod 64 = k \,\&\, 63.$$

To set this bit, we construct a temporary value with only the target bit active by shifting the integer constant 1 to the left by the bit position b, and then update the mask word using the bitwise OR:

$$B_w \leftarrow B_w \mid (1 \ll b).$$

To reconstruct candidate positions from the bitmask, we iterate over all 64-bit words and extract the indices of the set bits.

For a given word index w, let B_w again denote the corresponding 64-bit word. If $B_w = 0$, the word contains no valid candidates. Otherwise, we repeatedly extract the least significant set bit until B_w becomes zero.

The position of this bit within the word is obtained using the number of trailing zeros:

$$b = \mathrm{ctz}(B_w),$$

where ctz denotes the count of trailing zero bits.

The corresponding global candidate index is then

$$k = 64w + b.$$

After processing a candidate, the lowest set bit is cleared using the operation

$$B_w \leftarrow B_w \,\&\, (B_w - 1),$$

which removes the extracted bit and guarantees termination after all set bits have been processed.

This approach ensures that candidate extraction runs in time proportional to the number of valid candidates rather than the size of the bitmask.

3.2.3 Using a Bitmask for All Queries Together

Instead of allocating a separate bitmask for each query and clearing it after processing, the `candidate_v4` implementation uses single bitmask large enough to store candidate positions for all queries simultaneously. By restructuring the search to iterate primarily over the text followed by the queries, the implementation achieves a more cache-friendly access pattern while populating this unified bitmask.

For n queries, the total bitmask size becomes

$$h_{\text{total}} = n \cdot h,$$

where h is the number of 64-bit words needed for a single query.

To access the word corresponding to a candidate at position k for query q, we first compute the offset for that query:

$$l = q \cdot h.$$

The global word index in the combined bitmask is then

$$w_{\text{total}} = l + (k \gg 6),$$

where, as before, $k \gg 6$ gives the word index within the query's portion of the bitmask.

Reconstructing candidate positions proceeds in a similar manner. We iterate over all queries q, and within each query, over all words w from 0 to $h - 1$. For each word index, the offset-adjusted index is

$$w_{\text{total}} = l + w.$$

Candidate positions are then extracted from the set bits of $B_{w_{\text{total}}}$ using the same approach described previously.

3.3 Rolling-Hash

As one of several approaches, we also evaluate a Rabin–Karp–based rolling-hash in the `hash_v1` implementation (see Listing 3), which is widely used in text search due to its favorable average-case time complexity [3].

Let the query be a string of length M, with characters $q_0, q_1, \ldots, q_{M-1}$. Its hash is computed as

$$h(q) = \sum_{i=0}^{M-1} q_i \, B^{M-1-i},$$

where B is a prime number used as the base, and q_i is the numeric value of the i-th character.

Similarly, for a substring (window) of length M in the text starting at position j, with characters $t_j, t_{j+1}, \ldots, t_{j+M-1}$, the hash is

$$h_j = \sum_{i=0}^{M-1} t_{j+i} \, B^{M-1-i}.$$

To avoid recomputing h_j from scratch for each window, we apply the Rabin-Karp rolling hash formula:

$$h_{j+1} = \left(h_j - t_j \, B^{M-1}\right) B + t_{j+M}.$$

Whenever a window hash matches the query hash, a direct character-by-character comparison is performed to verify the match and eliminate potential hash collisions. This procedure is repeated independently for each query in the input set. All arithmetic is performed using 64-bit unsigned integers, implicitly modulo 2^{64}.

3.4 Grouping Queries by Length

The first implementation `hash_v1` scans the entire text separately for each query. Given a large text that exceeds the CPU's L1 and L2 cache capacities, this approach creates a performance bottleneck, since the text must be reloaded from RAM for every query.

To improve memory bandwidth utilization and temporal cache locality, `hash_v2` groups all queries by their length (see Listing 4). For each group of queries of the same length, a `std::unordered_map` is used as a hash table, mapping each query hash to the indices of all queries sharing that hash. This allows every text window of the corresponding length to be checked simultaneously against all relevant queries in constant average time. When a hash match occurs, the substring is verified against the original query to eliminate false positives caused by hash collisions. By grouping queries and using a hash table, the text is scanned only once per query length, rather than once per individual query, greatly reducing memory traffic and runtime. The rolling hash itself is computed as in `hash_v1`, using a precomputed power of B to efficiently update the hash from one window to the next.

```cpp
#define PRIME 1000003

uint64_t compute_power(const size_t length) {
    uint64_t p = 1;
    for (size_t i = 1; i < length; ++i) {
        p *= PRIME;
    }
    return p;
}

uint64_t hash_string(const char *s, const size_t length) {
    uint64_t h = 0;
    for (size_t i = 0; i < length; ++i) {
        h = h * PRIME + static_cast<unsigned char>(s[i]);
    }
    return h;
}

std::vector<std::vector<size_t>>
find_hash_v1(const std::string &text, const std::vector<std::string> &queries) {
    std::vector<std::vector<size_t>> indices(queries.size());
    const size_t text_length = text.length();
    for (size_t i = 0; i < queries.size(); ++i) {
        const std::string &query = queries[i];
        const size_t query_length = query.length();
        if (query_length == 0 || query_length > text_length) {
            continue;
        }
        const uint64_t query_hash = hash_string(query.data(), query_length);
        const uint64_t power = compute_power(query_length);
        uint64_t window_hash = hash_string(text.data(), query_length);
        for (size_t j = 0; j <= text_length - query_length; ++j) {
            if (window_hash == query_hash &&
                std::memcmp(text.data() + j, query.data(), query_length) == 0) {
                indices[i].push_back(j);
            }
            if (j < text_length - query_length) {
                window_hash -= static_cast<unsigned char>(text[j]) * power;
                window_hash = window_hash * PRIME + static_cast<unsigned char>(
                                                    text[j + query_length]);
            }
        }
    }
    return indices;
}
```

Listing 3: `hash_v1` implementation (`hash_v1_text_search.cpp`).

```cpp
std::vector<std::vector<size_t>>
find_hash_v2(const std::string &text, const std::vector<std::string> &queries) {
    std::vector<std::vector<size_t>> indices(queries.size());
    const size_t text_length = text.length();
    std::unordered_map<size_t, std::vector<size_t>> length_groups;
    for (size_t i = 0; i < queries.size(); ++i) {
        length_groups[queries[i].length()].push_back(i);
    }
    for (auto const &[query_length, query_indices] : length_groups) {
        if (query_length == 0 || query_length > text_length) {
            continue;
        }
        std::unordered_map<uint64_t, std::vector<size_t>> hash_table;
        for (size_t q : query_indices) {
            uint64_t h = hash_string(queries[q].data(), query_length);
            hash_table[h].push_back(q);
        }
        const uint64_t power = compute_power(query_length);
        uint64_t window_hash = hash_string(text.data(), query_length);
        for (size_t j = 0; j <= text_length - query_length; ++j) {
            auto it = hash_table.find(window_hash);
            if (it != hash_table.end()) {
                for (const auto q : it->second) {
                    if (std::memcmp(text.data() + j, queries[q].data(),
                                    query_length) == 0) {
                        indices[q].push_back(j);
                    }
                }
            }
            if (j < text_length - query_length) {
                window_hash -= static_cast<unsigned char>(text[j]) * power;
                window_hash = window_hash * PRIME + static_cast<unsigned char>(
                                                        text[j + query_length]);
            }
        }
    }

    return indices;
}
```

Listing 4: `hash_v2` implementation (`hash_v2_text_search.cpp`).

3.5 Evaluation

In Figure 1 and Figure 2, we observe that the `candidate_v1`, `candidate_v2`, and `hash_v1` variants consistently exhibit the lowest performance, independent of the number of books queried, the number of queries, or whether common or long words are used.

An exception occurs for low query counts, up to 35 common-word queries or 20 long-word queries, where `hash_v2` performs worse than the other variants, likely due to initialization overhead.

By contrast, the `std` variant consistently achieves the best performance across all test cases, which can be attributed to its implementation. In libstdc++, for example, `std::basic_string::find()` performs a search optimized with low-level memory operations, likely leveraging SIMD instructions on most architectures. Furthermore, the implementation differentiates between single-character searches and longer query strings, reducing overhead in common cases [4].

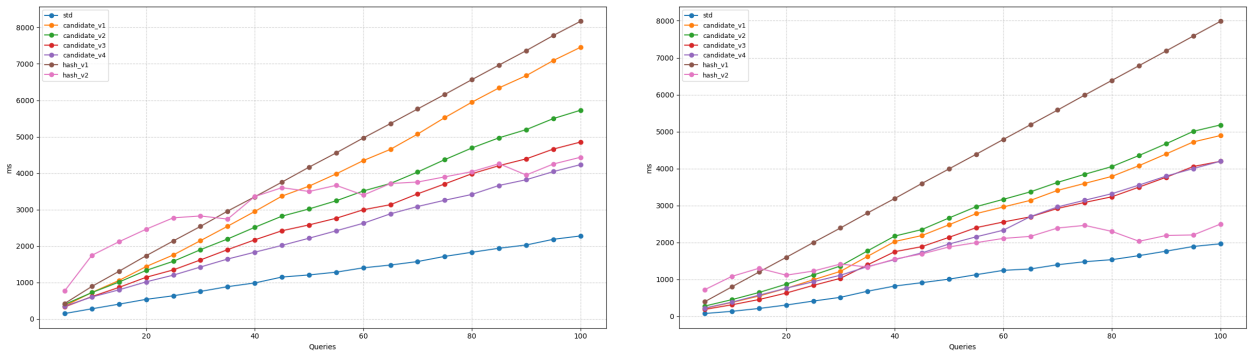
Our implementations, in contrast, do not employ length-dependent optimizations. All query strings are processed using a uniform search strategy, independent of their length. While this approach simplifies the code, incorporating query-length–specific optimizations similar to those in libstdc++ could further improve performance.

The bitmask-based approaches, `candidate_v3` and `candidate_v4`, show similar performance for long, less-common words. However, `candidate_v4` performs significantly better for common words.

This is due to different memory access patterns. In `candidate_v3`, the text is reloaded from main memory for every query, causing cache thrashing; the large text data constantly evicts the bitmask, which must be updated much more frequently for common words. In the `candidate_v4` approach, the text is scanned only once. This allows the bitmask to stay – at least partly – within the cache.

Finally, we observe that the `hash_v2` variant performs comparably to the bitmask-based candidate approaches when processing the common query set. However, for the long word queries, its efficiency becomes even more pronounced, trailing the `std` implementation by only a few hundred milliseconds.

This performance is due to the fact that updating and comparing the rolling hash window is a constant time operation that does not depend on the query length. Consequently, the more expensive byte-by-byte string verification and result vector access are only triggered in the event of a hash match. Since the long query set statistically results in significantly fewer matches, the overhead of these operations is largely avoided, allowing the algorithm to maintain high throughput.



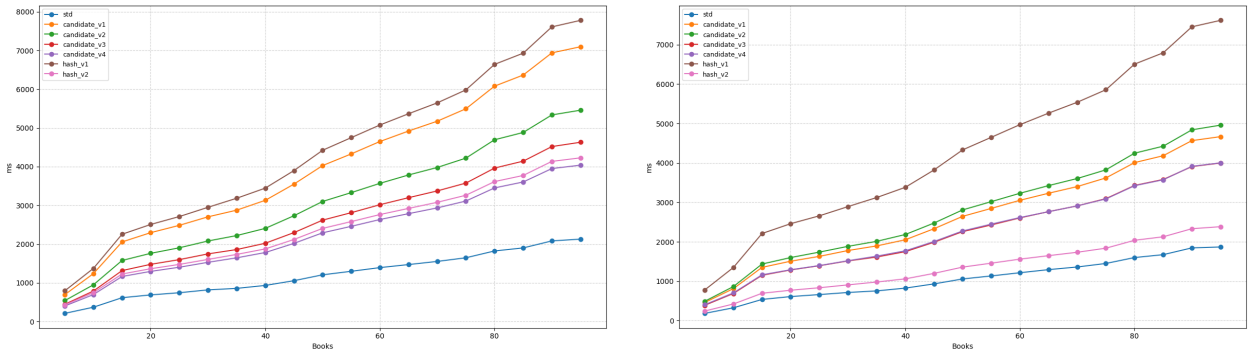
(a) Common words.

(b) Long words.

Figure 1: Runtime for sequential implementations for varying query counts (98 books).

4 Parallelization Across CPU Threads Using OpenMP

Having evaluated several sequential implementations, we now transition to parallelizing the most effective variants using OpenMP. This includes the best candidate selection and validation approaches, as well as the reference implementation. Additionally, we adapt the best-performing hash-based variant for parallel execution, which requires addressing its inherent data dependencies.



(a) Common words.

(b) Long words.

Figure 2: Runtime for sequential implementations for varying book counts (100 queries).

4.1 Candidate Selection and Validation

We use the two bitmask-based approaches, `candidate_v3` and `candidate_v4`, in the implementations `candidate_openmp_v1` and `candidate_openmp_v2`, respectively. The queries are distributed across all CPU threads for both the selection and validation phases.

In the approach that creates a bitmask for each query, a separate bitmask is generated for every query, instead of creating a single bitmask once and reusing it for all queries. When using a single large bitmask for all queries, the bitwise OR must be replaced with its atomic counterpart to avoid race conditions. OpenMP provides the directive `#pragma omp atomic` for this purpose [5]. This data dependency introduces a disadvantage compared to the per-query bitmask approach.

For the validation phase, no synchronization is required, since each query maintains its own vector within the overall result vector.

4.2 Rolling-Hash

The `hash_openmp` implementation parallelizes `hash_v2` by distributing independent query length groups across CPU threads. Parallelization is performed at the granularity of query groups sharing the same length, as only those can be processed together using a single rolling hash window.

Since OpenMP requires an integer index or a random access iterator for the `parallel for` directive, the `std::unordered_map` used in the sequential version cannot be parallelized directly [5]. Therefore, in a sequential preprocessing step, all query indices are stored in a vector and sorted by ascending query length. Based on this ordering, a second vector of metadata blocks is constructed, each storing a query length, the start position within the sorted index vector, and the number of queries in the group.

These blocks constitute independent work units and are processed in parallel using the OpenMP `parallel for` directive with dynamic scheduling [5]. Dynamic scheduling improves load balancing, as the computational cost varies with query length and the number of candidate matches in the text.

All auxiliary data structures required during the search phase, including the hash table mapping hash values to query indices, are allocated locally within the parallel region. As each query writes exclusively to its own result vector, the parallel execution requires no synchronization and is free of data races.

4.3 Reference Implementation

Finally, we also parallelized the reference implementation by distributing the queries across the threads. In the `std_openmp` implementation, using OpenMP, this modification required adding only a single line of code to the already very simple and concise reference implementation.

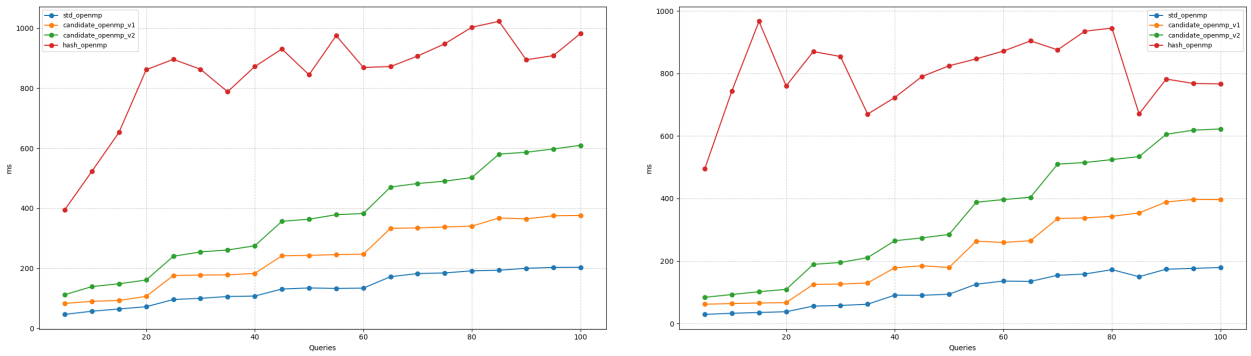
4.4 Evaluation

AAs illustrated in Figure 3 and Figure 4, the `candidate_openmp_v1` implementation outperforms `candidate_openmp_v2`. This performance gap is primarily due to the overhead of atomic operations required by the latter, which `candidate_openmp_v1` avoids. Despite this difference, both variants show significant scalability, achieving average speedups of 965% and 534% over their respective sequential counterparts.

The `hash_openmp` implementation shows the lowest performance among the parallel versions, achieving an average speedup of only 244% compared to the `hash_v2` variant. This is primarily due to inefficient load balancing: since the workload is partitioned by distinct query lengths, many threads remain idle if the queries are of uniform length.

In contrast, the candidate-based approaches parallelize across all queries, ensuring much higher thread utilization. Furthermore, the hash-based approach incurs a high computational overhead per text window, requiring rolling hash arithmetic and expensive hash map lookups. The candidate selection phase, however, relies on simple integer comparisons that are significantly more efficient to execute on modern CPUs.

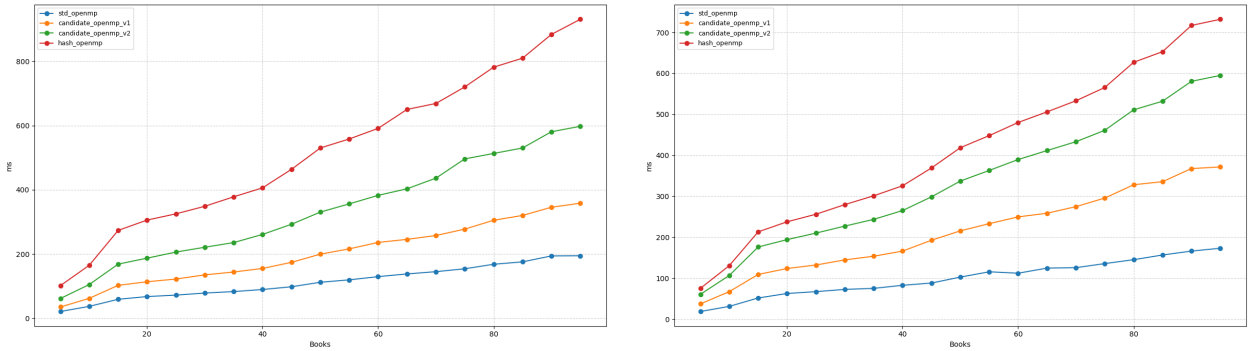
The parallelized reference implementation, `std_openmp`, delivers the highest overall performance, achieving an average speedup of 855% compared to its sequential counterpart. This result highlights a key takeaway: by leveraging the C++ Standard Library in conjunction with OpenMP, it is possible to develop a highly efficient text search with minimal implementation effort. The fact that a few lines of code can outperform more complex custom approaches underscores the maturity of modern library algorithms and their suitability for parallel execution.



(a) Common words.

(b) Long words.

Figure 3: Runtime for OpenMP implementations for varying query counts (98 books, 20 threads).



(a) Common words.

(b) Long words.

Figure 4: Runtime for OpenMP implementations for varying book counts (100 queries, 20 threads).

5 Parallelization Across the GPU Using OpenCL

All OpenCL implementations use a two-dimensional NDRange layout [6], where each work item processes one text position and one query. Work items are grouped into workgroups of 256 threads. GPUs execute threads in fixed-size SIMD groups rather than individually. On NVIDIA GPUs, these SIMD groups are called warps and consist of 32 threads, while on AMD GPUs the corresponding groups are called wavefronts and consist of 64 threads on GCN and CDNA architectures or 32/64 threads on newer RDNA architectures [7]. Consequently, a workgroup of 256 threads corresponds to 8 warps on NVIDIA GPUs and 4 wavefronts on AMD GPUs. Choosing a workgroup size that is a multiple of the hardware SIMD width is a common optimization practice and helps to efficiently utilize the GPU execution units [8], [9].

We did not parallelize rolling-hash-based implementations, since the inherent data dependencies in rolling hash computations complicate efficient task- or data-parallel execution. Consequently, we focus on parallelizing the candidate-based approaches.

5.1 Using a Bitmask

In the `candidate_opencl_v1` implementation, a single global bitmask is used to store all potential matches for all queries across the entire text. Instead of separating candidate selection and validation into two kernels, each kernel invocation directly validates a single query at a single text position and writes the result to the corresponding bit in the mask (see Listing 5). The candidate selection conditions are reused as a pre-filter within the kernel to reduce unnecessary full comparisons.

While this approach simplifies the kernel structure, it introduces several performance bottlenecks:

- The bitmask must be transferred between host and device memory. Since the text itself already needs to be copied to the GPU, this additional transfer increases the overall data movement overhead.
- Updates to the bitmask require atomic bitwise OR operations to avoid race conditions, which can significantly limit scalability.
- After kernel execution, the entire bitmask must be unpacked to reconstruct the result vectors. This post-processing step has unfavorable time complexity and can become costly for large inputs.

5.2 Two-Phase Approach

To improve performance and better utilize GPU work groups, we employ a two-phase strategy for the `candidate_opencl_v2` implementation:

1. In the first phase, we count the number of candidate matches for each query within each work group. This allows us to determine the exact size of the result buffer and the offsets for each work group's results.
2. In the second phase, candidates are validated and the confirmed matches are written into their designated positions within the shared result buffer.

Both phases are implemented within a single kernel. A mode variable is used to distinguish between counting and validation operations (see Listing 6).

Counting is performed per work group to minimize synchronization and reduce the need for atomic operations.

After counting, a single result buffer is allocated with partitions for each work group according to the counts computed in the first phase, and the counts buffer is reset on the GPU.

During validation, each thread examines a specific text position and query pair, applying the same pre-filtering strategy used during counting. The counts buffer is then reused to track insertion positions within each work group's partition of the result buffer, allowing efficient parallel writing of validated matches.

```
__kernel void test_candidates(
    __global uint *mask,
    uint group_amount,
    __global const char *text,
    uint text_length,
    __constant const char *flatten_queries,
    __constant const uint *query_offsets,
    __constant const uint *query_lengths
) {
    const uint i = get_global_id(0);
    const uint j = get_global_id(1);
    if (i >= text_length) {
        return;
    }
    const uint query_offset = query_offsets[j];
    const uint query_length = query_lengths[j];
    if (i + query_length > text_length) {
        return;
    }
    const uint k = get_local_id(0);
    const uint l = get_group_id(0);
    const uint mid = query_length >> 1;
    const uint end = query_length - 1;
    bool candidate = text[i] == flatten_queries[query_offset] &&
                     text[i + mid] == flatten_queries[query_offset + mid] &&
                     text[i + end] == flatten_queries[query_offset + end];
    if (candidate) {
        bool match = true;
        for (uint m = 1; m < query_length - 1; ++m) {
            if (m == mid) {
                continue;
            }
            if (text[i + m] != flatten_queries[query_offset + m]) {
                match = false;
                break;
            }
        }
        if (match) {
            atomic_or(&mask[(j * group_amount * 8) + (l * 8) + (k >> 5)],
                                                    (uint)1 << (k & 31));
        }
    }
}
```

Listing 5: OpenCL Kernel for the `candidate_opencl_v1` implementation (`candidate_opencl_v1_text_search.cpp`).

```
__kernel void count_and_test_candidates(
    __global const char *text,
    __global uint* counts,
    __global uint* results,
    uint text_length,
    __constant const char *flatten_queries,
    __constant const uint *query_offsets,
    __constant const uint *query_lengths,
    __global const uint *result_offsets,
    uint mode
) {
    const size_t i = get_global_id(0);
    const size_t j = get_global_id(1);

    if (i >= text_length) {
        return;
    }
    const uint query_offset = query_offsets[j];
    const uint query_length = query_lengths[j];
    if (i + query_length > text_length) {
        return;
    }
    const uint mid = query_length >> 1;
    const uint end = query_length - 1;
    bool candidate = text[i] == flatten_queries[query_offset] &&
                     text[i + mid] == flatten_queries[query_offset + mid] &&
                     text[i + end] == flatten_queries[query_offset + end];
    if (candidate) {
        const uint index = j * get_num_groups(0) + get_group_id(0);
        if (mode == 1) {
            atomic_inc(&counts[index]);
        } else {
            bool match = true;
            for (uint k = 1; k < query_length - 1; ++k) {
                if (k == mid) {
                    continue;
                }
                if (text[i + k] != flatten_queries[query_offset + k]) {
                    match = false;
                    break;
                }
            }
            if (match) {
                uint count = atomic_inc(&counts[index]);

                results[result_offsets[index] + count] = (uint)i;
            }
        }
    }
}
```

Listing 6: OpenCL Kernel for the `candidate_opencl_v2` implementation (`candidate_opencl_v2_text_search.cpp`).

5.3 Estimating the Candidates

Instead of counting candidate matches on the GPU, we estimate the required size of the result buffer on the CPU beforehand in the `candidate_opencl_v3` implementation. We employ a heuristic based on query length and typical character frequencies in English:

- For queries of length 1, we add 12.7% of the text size, reflecting the frequency of the most common letter, *e* [10].
- For queries of length 2, we add 3.56% of the text size, reflecting the frequency of the most common bigram, *th* [11].
- For queries of length 3, we add 1.81% of the text size, corresponding to the frequency of the most common trigram, *the* [12].
- For longer queries, we add 0.1% of the text size for each query.

Since this estimation may result in over-allocation, the final buffer size is halved to balance memory usage and the risk of overflow, and then capped at the maximum OpenCL buffer allocation size to ensure compatibility.

During kernel execution, we use a single global result buffer and a single global counter buffer. The counter is atomically incremented by each thread to assign a unique position for each match in the result buffer. Additionally, a second buffer stores the mapping from result indices to query indices, allowing reconstruction of which query produced each match.

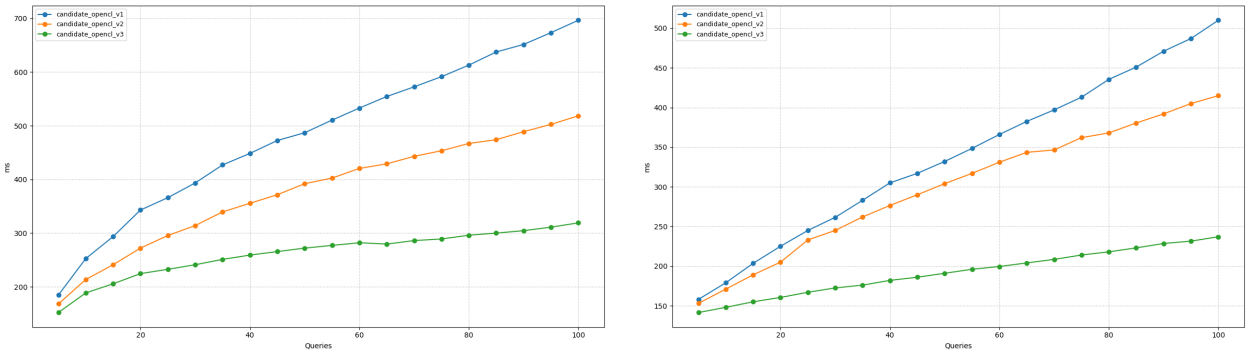
If the allocated buffer proves insufficient during execution, excess matches are discarded to prevent buffer overflow.

5.4 Evaluation

As shown in Figure 5 and Figure 6, the bitmask approach exhibits the poorest performance due to its inherent bottlenecks, achieving an average speed-up of 433% over its sequential equivalent. The two-phase strategy demonstrates improved performance across all test cases, yielding an average speed-up of 545%.

By employing a heuristic to estimate the required result buffer size, the `candidate_opencl_v3` implementation achieves significantly better performance with an average speed-up of 915%. However, this remains below the performance of the reference OpenMP implementation.

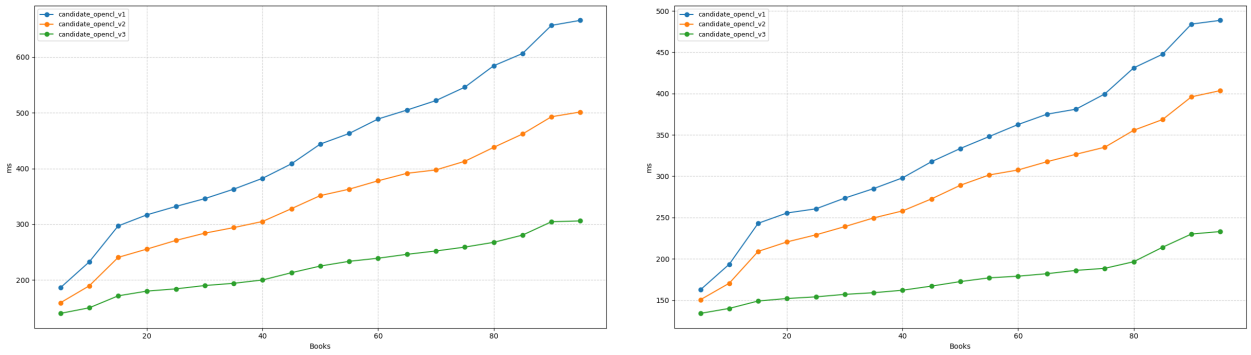
The heuristic proves sufficient for most real-world use cases and performs adequately even in our extreme test scenarios. For applications involving exceptionally common queries where guaranteed correctness is critical, the `candidate_opencl_v2` implementation with its two-phase counting approach provides a more conservative alternative at the cost of reduced performance.



(a) Common words.

(b) Long words.

Figure 5: Runtime for OpenCL implementations for varying query counts (98 books).



(a) Common words.

(b) Long words.

Figure 6: Runtime for OpenCL implementations for varying book counts (100 queries).

6 Parallelization Across Processes Using MPI

7 Conclusion

References

[1] Project Gutenberg. "Project Gutenberg," Accessed: Feb. 2, 2026. [Online]. Available: `https://www.gutenberg.org/`

[2] Wikipedia contributors. "Most common words in English — Wikipedia, the free encyclopedia," Accessed: Feb. 2, 2026. [Online]. Available: `https://en.wikipedia.org/wiki/Most_common_words_in_English`

[3] R. M. Karp and M. O. Rabin, "Efficient randomized pattern-matching algorithms," *IBM Journal of Research and Development*, vol. 31, no. 2, pp. 249–260, 1987. DOI: `10.1147/rd.312.0249`

[4] Free Software Foundation, *The GNU C++ Library*, version 15.2.0, Files: libstdc++-v3/include/bits/basic_string.h, libstdc++-v3/include/bits/basic_string.tcc, libstdc++-v3/include/bits/char_traits.h. Accessed: Feb. 2, 2026. [Online]. Available: `https://gcc.gnu.org/git/?p=gcc.git;a=tree;h=refs/tags/releases/gcc-15.2.0`

[5] OpenMP Architecture Review Board, *OpenMP Application Programming Interface*, version 6.0, 2024. Accessed: Feb. 2, 2026. [Online]. Available: `https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-6-0.pdf`

[6] Khronos OpenCL Working Group, *The OpenCL Specification*, version 3.0, 2025. Accessed: Feb. 2, 2026. [Online]. Available: `https://registry.khronos.org/OpenCL/specs/3.0-unified/html/OpenCL_API.html`

[7] Advanced Micro Devices, Inc., *HIP documentation*, version 7.2.0, 2025. Accessed: Feb. 2, 2026. [Online]. Available: `https://rocm.docs.amd.com/projects/HIP/en/docs-7.2.0/`

[8] NVIDIA Corporation, *OpenCL Best Practices Guide*, 2011. Accessed: Feb. 2, 2026. [Online]. Available: `https://developer.download.nvidia.com/compute/DevZone/docs/html/OpenCL/doc/OpenCL_Best_Practices_Guide.pdf`

[9] Advanced Micro Devices, Inc., *AMD APP SDK OpenCL Optimization Guide*, 2015. Accessed: Feb. 2, 2026. [Online]. Available: `https://docs.amd.com/v/u/en-US/AMD_OpenCL_Programming_Optimization_Guide2`

[10] Wikipedia contributors. "Letter frequency — Wikipedia, the free encyclopedia," Accessed: Feb. 2, 2026. [Online]. Available: `https://en.wikipedia.org/wiki/Letter_frequency`

[11] Wikipedia contributors. "Bigram — Wikipedia, the free encyclopedia," Accessed: Feb. 2, 2026. [Online]. Available: `https://en.wikipedia.org/wiki/Bigram`

[12] Wikipedia contributors. “Trigram — Wikipedia, the free encyclopedia,” Accessed: Feb. 2, 2026. [Online]. Available: https://en.wikipedia.org/wiki/Trigramh